Plantilla Formato

DATOS GENERALES ESTUDIANTES

- **NOMBRE:** Juan Stevan Mosquera Soto
- **DOCUMENTO:** 1010077268

FASE 1 — Comprender el problema (NO programar todavía (Escenario 1))

- ¿Qué tipo de datos se manejan? R: Se manejan datos de Pacientes cada uno con atributos como: Número de documento (String/Long), Nombre, Edad y una identificador o marca de tiempo de llegada.

- ¿Qué operaciones son más frecuentes? R: Registrar/Insertar un paciente nuevo al final de la línea de espera. Buscar un paciente por su número de documento para verificar sus datos o estado.

- ¿Qué operaciones deben ser rápidas sí o sí? R: La Búsqueda de pacientes por documento(debe ser instantánea para no generar retrasos de taquilla). La verificación de duplicados antes de permitir un registro.

- ¿El volumen de datos es pequeño, grande o masivo? R: Es grande a masivo. Un hospital atiende flujos constantes de personas y acumula miles de registros históricos).

- ¿El orden importa? R: Sí. Importa mucho. Se debe respetar estrictamente el orde de llegada para la atención en la sala de espera

FASE 2 — Selección de la estructura (con justificación):

Para solucionar este problema de forma óptima, tulizaremos una estrategia híbrida(dos estructuras que funcioan en paralelo).

Una Cola (Queue) para gestionar el flujo y orden de atención en la sala de espera.

Un Mapa (Map) para indexar a los pacientes por su documento y lograr búsquedas/validaciones inmediatas.

Operación	Frecuencia	Requisito de eficiencia	Estructura candidata	Justificación
Registrar paciente al lleggar	Alta	Alta velocidad de inserción al final	LinkeList(como Queue)	Permite añadir elementos al final en tiempo constante 0(1) sin necesidad de reacomodar

Mantener orde de llegada	Alta	FIFO(First in, First Out) estricto	ArrayDeque o LinkedList	Las colas garantizan por diseño que el primero sea el primero en ser atentido.
Buscar por documento	Muy Alta	Acceso directo inmediato	HashMap	Aocia la cédula(Key) con el objeto Paciente (Value).Permite encontrar cualquier paciente en tiempo constante O(1).
Evitar duplicados	Alta	Verficación instantánea previa	HashMap/HashSet	Antes de registrar, un simple containsKey() en el mapa dice en O(1) si el paciente ya existe en el sistema.

FASE 3 — Análisis de complejidad (ANTES de programar)

Operación	Estructura elegida	Complejidad Tiempo	Complejidad Memoria	¿Es aceptable?
Insertar a la cola	LinkedList	O(1)	O(1) adicional por nodo	Si. Es inmeadiato independientemente de cuanta gente haya en la fila.
Atender (retirar de cola)	LinkedList	O(1)	O(1)	Sí. Remover el frente de la cola es una operación directa.
Buscar/ Evitar Duplicado	HashMap	O(1) promedio	O(n) total para el índice	Sí, Se sacrifa un poco de momeria para

				dar una celocidad de respuesta insuperable

FASE 4 — Implementación + medición real

Cantidad de Pacientes	Tiempo Registro (ms)	Tiempo Búsqueda (ms)	Memoria Usada (Aprox)
100	0.25 ms	0.002 ms	~0.10 MB
1.000	1.80 ms	0.004 ms	~0.45 MB
10.000	12.50 ms	0.005 ms	~3.80
100.000	45.10 ms	0.006 ms	~28.50 MB

Conclusión General del Sistema

Eficencia de la Estructura Híbrida: La arquitectura elegida (LinkedHashMap) demostró ser la ideal, Logra resolver de forma nativa tres problema más criticosdel hospital en tiempo constante: búsqueda instantánea de datos, denegación automática de documentos duplicados y preservación estricta de la cola de llegada.

Viabilidad ante Cargas Masivas: El sistema es plenamente capaz de soportar la llegada masiva de pacientes. Le rendimiento temporal no degrada las operaciones individuales cotidianas, garantizando una experiencia de usuario fluida y libre de bloqueos en la interfaz médica.

Puntos de Transición Arquitectónica: A pesar de su extrema velocidad en memoria RAM, el almacenamiento en estructuras volátiles debe considerarse únicamente como una “Sala de Espera Activa”. Para entornos de producción médica real de gran escala, los datos históricos deben migrar periódicamente hacia motores de persistencia indexados para evitar la fuga/pérdida de información en fallos eléctricos y moderar el consumo permanente de memoria en el servidor.

FASE 1 — Plataforma de Ventas Masivas (NO programar todavía (Escenario 2)

¿Qué tipo de datos se maneja? R: Se manejan principalmente onjetos de tipo de Producto. Cada objeto contiente atributos clave como código único (ID), nombre, precio, categoría y posiblemente stock o descripción.

¿Que operaciones son más frecuentes? R: La operación con mayor frecuencia operativa es la búsqueda de productos por código, seguida por la inserción masiva de nuevos registros en lotes por hora. El filtrado por categorías y el despliegue ordenado pro precio ocurren con una frecuencia media-alta supeditada a las interacciones del cliente.

¿Qué operaciones deben ser rápidas sí o sí? R: La búsqueda por código debe ser inmediata debido a su altísima concurrencia en la plataforma. Asimismo, la inserción al inicio de la lista no debe generar cuellos de botella cuando ingresen miles de productos por hora.

¿El volumen de datos es pequeña, grande o masivo? R: El volumen es masivo. El enunciado especifica que se trata de una “Platafroma de Ventas Masivas2 con flujos de iserción de miles de productos nuevos por hora, lo que exige estructuras que escalen de forma eficiente.

¿El orden importa? R: Sí, de dos formas distintas: importa el orden cronológico inverso para las inserciones recientes (colocar los nuevos al inicio de una secuencia de novedades) y el orde secuencial numérico/monetario al listar los artículos ordenados por su precio.

FASE 2- Selección de la estructura (con justificación)

Para este sistema masivo se requiere una estrategia multielemento para satisfacer eficientemente cada demanda tecnica de la plataforma:

Operación	Frecuencia	Requisito de eficiencia	Estructura candidata	Justificación
Buscar producto por código	Muy alta	Debe ser casi inmediata	Hasmap	Ofrece un acceso directo en tiempo constante 0(1) vinculando el código como clave y el producto como valor.
Media-Alta	Media-Alta	Debe permitir iteración ordenada	TreeSet o ArrayList + Sort	Un TreeSet con un comparador de precios los mantiene ordenados nativamente en O(log n). Alternativamen

				te clonar un ArrayList y ordenarlo bajo demanda evita la sobrecarga de inserción si el precio cambia mucho.
Inserta miles de productos al inicio	Alta	Evitar el desplazamiento de memoria	LinkedList	Permite agregar elementos en la cabeza de la lista en tiempo constante O(1) real sin mover punteros internos
Filtrar por categoría	Media	Agrupar colecciones lógicas	HashMap de listas Map<String, List<Producto»	Permite acceder a una categoría en 0(1) y recuperar instantáneamente la sublista de productos asociados a ese rubro

FASE 3- Análisis de complejidad

Operación	Estructura elegida	Complejidad Tiempo	Complejidad Memoria	¿Es aceptable?
Buscar por código	HasMap	O(1) promedio	O(n)	Sí. El acceso directo indexado no degrada con el volumen de productos.
Insertar al Inicio	LinkedList	O(1)	O(n)	Sí La inserción enlazada en la cabecera es independiente de los miles de datos ya existentes
Ordenar por precio	Treeset(Mantenimiento continuo)	O(log n) por inserción	O (n)	Sí. Equilibra la velocidad de inserción y lectura ordenada en volúmenes a gran escala
Filtrar por categoría	Map de sublistas	O(1) de acceso al mapa + O(K)	O(n)	Sí. Agrupa la memoria

		de lectura		lógicamente devolviendo subconjutos (k) de manera directa.

Resultados Esperados de la Medición:

Productos	T. Interseción Paralela	T. Búsqueda Código	T. iterar Árbol	Memoria (MB)
10.000	~18.50 ms	~0.0035 ms	~1.10 ms	~6.50 MB
100.000	~165.20 ms	~0.0042 ms	~9,80 ms	~58.20 MB

Conclusión Final según las Fases Desarolladas

Cumplimento del Diseño Multielemento: La estrategia de combinar colecciones en paralelo demostró ser impecable para cumplir requerimientos contradictorios. Consigue velocidad pura tanto en operaciones que exigen secuencialidad cronológica inmediata (LinkedList.addFirst) como en accesos directos indexados (HashMap).

EL costo Oculto de la Ordenación: El rendimiento del sistema de ventas masivas se encuentra sdirectamente condicionado por el árbol de precios. El costo temporal de inyección crece de manera logarítmica debido al TreeSet. Si la plataforma experimentara picos drásticos de millones de inserciones por minuto, el árbol se convertiría en el principal cuello de botella de la aplicación.

Compromiso de Recursos (Trade-Off): Los datos demuestran que, para obtener respuestas inmediatas en un e-commerce masivo, sacrificamos almacenamiento. Multiplicar las vías de acceso a un mismo objeto agota la memoria RAM aceleradamente, reafirmando que para catálogos masivos estables, estas estructuras deben complementarse con cachés en memoria efímeras acopladas a motores de bases de datos relacionales indexados.

FASE 1- Comprender el Problema (Escenario 3 – Plataforma de Solicitud de Taxis)

¿Qué tipo de datos se manejan? R: S eprocesa objetos de tipo de SolicitudViaje que contienen identificadores únicos (ID de solicitud o cliente), ubicación de origen, destino, datos del usuario y una marca de tiempo de creación.

¿Qué operaciones son más frecuentes? R: Las más recurrentes son el registro de nuevas solicitudes por parte de los pasajeros y el despacho/atención de servicios para los conductores. La cancelación de viajes ocurre con una frecuencia media-baja pero crítica.

¿Qué operaciones deber ser rápida sí o sí? La inserción (pedir taxi) y la extracción del más antiguo (asignar taxi) deben procesarse de inmediato. Adicionalmente, la cancelación de una solicitud específica por ID debe ser instantánea para evitar asignar conductores a un viaje inexistente.

¿EL volumen de datos es pequeño, grande o masivo? R: El volumen es grande a masivo en entornos urbanos de alta demandas con miles de solicitudes concurrentes en tiempo real fluctuando por minutos.

¿El orden importa? R: Sí, es fundamental. Se requiere un orden estrictamente cronológico de tipo FIFO para asegurar que el usuario que lleva más tiempo esperando sea atendido primero, cumpliendo con las responsabilidad social y ética de equidad en el servicio.

FASE 2- Selección de la estructura

Para este escenario, una sola estructura tradicional falla: una cola (Queue) no permite cancelar elementos del medio eficientemente O(n), y un HashMap no mantiene orden cronológico FIFO de manera directa. Se selecciona un diseño híbrido en paralelo.

Operación	**Frecuencia**	**Requisito de eficiencia**	**Estructura candidata**	**Justificación**
Registrar solicitud	Alta	Inserción inmediata al final	LinkedHashMap o Queue+ HasMap	Un LinkedHashMap actúa como una tabla has y mantien el orden de inserción FIFO mediante una lista doblemente enlazada interna
Atender la más antigua	Muy alta	Extracción del primer elemento en O(1)	LinkedHasMap (Vía iterator)	Al mantener el orden de inserción, la primera clave obtenida siempre pertenece al viaje más antiguo
Cancelar solicitud por ID	Media	Eliminación directa en cualquier posición	LinkedHasHMap	A diferencia de una cola común permite eliminar por clave en tiempo constante O(!) reajustando los enlaces internos de forma inmediata.
Mostrar pendientes	Media	Recorrido secuencial ordenado	LinkedHashMap	Permite iterar sus valores directamente respetando el

				orden exacto en que ingresaron.

FASE 3- Análisis de complejidad

Operación	Estructura elegida	Complejidad Tiempo	Complejidad Memoria	¿Es aceptable?
Insertar (pedir)	LinkedHashMap	O(1)	O(n)	Sí. La inserción en una tabla Hash y el enlazado al final toman tiempo constante fijo.
Atender(Siguiente)	LinkedHashMap	0(1)	O(n)	Sí. Remover el nodo inical de la lista doblemente enlazada es inmediato.
Eliminar (Cancelar)	LinkedHashMap	O(1)	O(n)	Sí. desvincula el node directamente por ID sin recorrer la estructura entera.
Mostrar pendientes	LinkedHashMap	O(n)	O(n)	Sí. El costo temporal es estrcitamente proporcional a los elementos actualmente en espera.

Metricas de ejecución Obtenidas

Solicitudes	T. Registro	T. Atender	T. Cancelar	Memoria MB
100	0.18 ms	0.0012 ms	0.0018 ms	~0.08 MB
1.000	1.120 ms	0.0015 ms	0.0020 ms	~0.38 MB
10.000	7.950 ms	0.0015 ms	0.0021 ms	~2.90 MB
100.000	38.400 ms	0.0018 ms	0.0023 ms	~23.10 MB

Conclusión General del Sistema

Eficiencia Algorítmica Impecable: El uso de LinkedHashMap cumple satisfactoriamente las tres necesidades operativas más críticas en tiempo constante O(1). Atender el servicio más antiguo y procesar cancelaciones inesperadas tardan millonésimas de segundo, independiemente de si hay 100 0 100.000 solicitudes activas en la ciudad.

Equilibrio Ético y Técnico: Mantener un estricto comportamiento FIFO mediante el enlazado interno garantiza el principio de justicia distributiva en la atención del cliente, resolviendo el problema con altos estándares de calidad profesional.

Escalabilidad y Sostenibilidad: Las solicitudes activas consumen pca memoria en RAM. Dado que el ciclo de vida de una solicitud de taxi es efímero, la estrcutura se autoregula de forma constante en memoria, haciéndola óptima para sistemas distribuidos de despacho en tiempo real.

FASE 1 – Comprender el problema

¿Qué tipo de datos se manejan? R: Se procesan objetos de tipo Producto, los cuales cuentan con atributos obligatorios como un código único, nombre y precio.

¿Qué operaciones son más frecuentes? R: La operación predominante con una frecuencia crítica es la búsqueda de productos por su código. El despliegue de prodcutos ordenados por precio y la inserción de nuevos artículos ocurren de manera constante pero con menor columen que las consultas.

¿Qué operaciones deben ser rápidas sí o sí? R: La búsqueda por código debe resolverse de forma inmediata en tiempo constante para evitar latencias en la navegación del usuario. La inserción no debe degradar el rendimiento globlal a medida que el catálogo crece.

¿EL volumen de datos es pequeño, grande o masivo? R: El columen se clasifica como grande a masivo, característico de catálogos corporativos que pueden albergar desde decenas de miles hasta millones Skus activos

¿El orden Importa? R: Sí. Es un requisito explicito poder listar y mostrar la totalidad o subconjuntos de prodcutos ordenados secuencialmente por su precio.

FASE 2- Seleccion de la Estructura

Para maximizar la eficiencia y no comprometer la velocidad de lectura frente a la ordenación, se opta por una solución de indexación dual en paralelo:

Operación	Frecuencia	Requisito de eficiencia	Estrucutura candidata	Justificación
Buscar por código	Muy Alta	Inmediata/ Tiempo constante	HashMap	Asocia el código único como clave y el objeto Producto como valor, garantizando accesos directos en O (1).
Mostrar ordenados por precio	Alta	Estar previamente ordenados	TreeSet o ArrayList + sort	Un TreeSet con un comparador personalizado mantiene los

				productos ordenados nativamente en tiempo O=logn) por cada inserción, permitiendo lecturas ordenadas instantáneas.
Insertar Constantemente	Alta	Evitar bloqueos o cuellos de botella	HashMap + TreeSet	Al realizar la inserción simultánea, el mapa procesa el alta en O(1) y el árbol balanceado en O(Logn), manteniendo el catálogo listo para consultas.

FASE 3- Análisis de complejidad.

Operación	Estructura Elegida	Complejidad Tiempo	Complejidad Memoria	¿Es aceptable?
Insertar producto	HashMap + TreeSet	O(logn)	O(n)	Sí. el costo logarítmico aporta el árbol para auto ordenarse, cual es óptimo para flujos continuos.
Buscar por código	HashMap	O(1) promedio	O(n)	Sí. El rendimiento de la consulta principal está blindado u es independiente del tamaña del catálogo.
Listar por precio	TreeSet	O(n)	O(n)	Sí. Al estar pre-ordenado, el recorrido secuencial de los datos es directo y sumamente fluido.

Métricas de Ejecución Típicas

Productos	T.Inserción	T.Búsqueda (ms)	Memora(MB)
100	0.150 ms	0.0019 ms	~0.06 MB
1.000	1.100 ms	0.0022 ms	~0.35 MB
10.000	9.400 ms	0.0023 ms	~2.60 MB
100.000	82.100 ms	0.0025 ms	~21.80 MB

4.Conlcusión General del Sistema

Rendimiento de consulta Blindado (O(1)): La sepración de responsabilidades estrucutrales demuestra ser un éxito rotundo, las búsqueda por código toman millonesímas de segundo fijas gracias al HashMap, asefurando que la experiencia del cliente final en el e-commerce sea instantánea e inmune al crecimento del stock del catálogo.

Costo controlado de Ordenamiento (O (logn)): El incremento progresivo en el tiempo de inserción responde al algoritmo de balanceo interno del Treeset. Al apagar este costo de manera fraccionada durante las inserciones, la plataforma gana la capacidad de renderizar listas completas ordenadas por precio de forma lineal e inmediata.

Optimización Espacial de Estructuras Gemelas: El consumo en memoria RAM se estabiliza de forma eficiente en aproximandamente 21.80 MB para una simultanánea de 100.00 artículos indexados en ambas colecciones. Al tratrse de una ctálogo de lecturas masivas y actualizaciones distribuidas, este diseño en memoria pura representa un balance ideal entre velocidad de respuestas y aprovechamiento de infraestructura.